BLUEBRIDGE: A LOW-LATENCY WIRELESS AUDIO STREAMING SYSTEM FOR USING A SMARTPHONE AS A PC SPEAKER

Project Report | Author: [Student Name] | Affiliation: [Department, Institution]

Abstract—This paper presents BlueBridge, a Windows-hosted application that captures the system audio output and streams it to a smartphone so that the phone can operate as a wireless speaker. The primary implementation uses Windows Audio Session API (WASAPI) loopback capture through the SoundCard Python library, NumPy-based conversion of floating-point samples to signed 16-bit pulse-code modulation (PCM), and an asynchronous aiohttp server that distributes binary audio frames over WebSocket connections. A mobile browser receives the PCM stream and schedules playback through the Web Audio API. The network path can use Bluetooth Personal Area Networking (PAN) or a local Wi-Fi network; a separate optional Android implementation demonstrates direct Bluetooth RFCOMM/Serial Port Profile (SPP) reception and playback through Android AudioTrack. The desktop interface provides QR-based connection assistance, network-interface selection, source-device selection, pause/resume control, and an RMS audio-level indicator. The supplied configuration specifies 44.1-kHz, 16-bit PCM and 2,048-frame capture blocks. Theoretical payload rates are 1.4112 Mbit/s for stereo and 0.7056 Mbit/s for mono, excluding transport overhead. The design dossier estimates an end-to-end latency budget of 34–69 ms; this is treated as a design estimate rather than a measured result because no reproducible latency-test dataset was supplied. The report also identifies implementation constraints and proposes optional multi-device connection management and synchronized multi-room playback as future work.

Index Terms—WASAPI loopback, WebSocket, Bluetooth PAN, RFCOMM, PCM audio, Web Audio API, Android AudioTrack, multi-device streaming.

I. INTRODUCTION

Audio output from a personal computer is commonly delivered through built-in speakers, wired headphones, or a dedicated wireless audio device. These options may be inconvenient when the PC speakers are unavailable, when a listener wants to move around a room, or when a spare smartphone could be reused as an audio endpoint. Standard Bluetooth audio sink support is not consistently exposed by desktop and mobile operating systems as a simple universal receiver workflow. Many software alternatives also depend on installing companion applications or using a shared Wi-Fi network.

BlueBridge addresses this problem by capturing the Windows system output and delivering it to a smartphone over a local IP connection. The phone can use a browser-based player without installing a dedicated application. Bluetooth PAN provides an alternative local network path when Wi-Fi is unavailable. An optional native Android client uses Bluetooth RFCOMM/SPP as a separate direct-socket approach.

The principal contributions of the project are: (1) a Windows loopback capture and PCM conversion pipeline; (2) asynchronous WebSocket distribution to browser clients; (3) a mobile web player with playback controls and buffering presets; (4) a desktop dashboard with QR-based connection assistance; and (5) an optional native Android receiver. The project further identifies multi-device connection and synchronized playback as a future enhancement.

II. PROBLEM STATEMENT AND OBJECTIVES

The problem is to provide a practical method for forwarding the audio mix rendered by a Windows PC to a smartphone with low additional delay, minimal setup, and no mandatory cloud service. The system should support a local Wi-Fi connection and a Bluetooth PAN connection, present a simple connection workflow, and avoid allowing temporary network delay to create an indefinitely growing audio backlog.

A. Objectives

- Capture the active Windows output endpoint using loopback recording.
- Convert captured samples into a compact, widely supported PCM representation.
- Transmit audio as binary frames through an asynchronous local server.
- Provide a browser receiver with start/stop, volume, channel-mode, and buffering controls.
- Offer QR-code and IP-address assistance through a desktop GUI.
- Provide an optional Android RFCOMM/SPP receiver using AudioTrack.
- Document measurable performance requirements and a roadmap for optional simultaneous multi-device playback.

B. Scope

The primary, documented user path is Windows PC to one or more mobile browser clients over Bluetooth PAN or Wi-Fi. The native Android SPP path is an optional alternative and is not the same transport as the browser WebSocket path. iOS support is represented by the browser client; the supplied native Android code does not establish a native iOS implementation.

III. RELATED TECHNOLOGIES

WASAPI is the Windows audio subsystem interface used to access audio endpoints. In loopback mode, an application records the stream being rendered by an output endpoint rather than recording microphone input. SoundCard provides a Python interface for speaker enumeration and loopback recording. NumPy supports vectorized signal conversion and channel averaging.

WebSocket provides a persistent, bidirectional connection over a TCP-based transport and supports binary messages, making it suitable for delivering PCM chunks without repeatedly creating HTTP requests. In the browser, the Web Audio API provides an audio processing and playback graph. The supplied client uses scheduled AudioBuffer playback and buffering presets. On Android, BluetoothSocket provides RFCOMM socket communication and AudioTrack provides a low-level audio playback interface with a low-latency performance mode where supported by the device.

These technologies form the basis of the implementation described in the supplied BlueBridge engineering dossier and source excerpts. This report does not claim that they guarantee a particular latency on every device or Bluetooth adapter.

IV. SYSTEM ARCHITECTURE

BlueBridge is organized into six functional layers: audio capture, digital audio processing, network transport, desktop control, mobile web playback, and the optional native Android companion. The Windows host captures the master output mix. AudioStreamer converts and distributes frames. The aiohttp server manages browser sessions and serves the web interface. The desktop GUI exposes connection and monitoring functions. A phone browser decodes and schedules the incoming PCM stream.

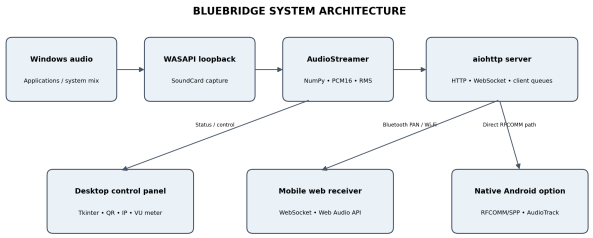


Fig. 1. Functional architecture of the BlueBridge system.

A. Audio Capture Layer

The audio capture engine selects either a specified speaker endpoint or the default speaker through SoundCard. It obtains a loopback recording interface associated with the selected output device and reads fixed-size blocks in a dedicated worker thread. This allows audio capture to proceed independently of the GUI and asynchronous server event loop.

B. Processing and Distribution Layer

Captured samples are represented as floating-point arrays. The implementation computes RMS level as the square root of the mean squared sample amplitude. It then clips and scales the samples into signed 16-bit integers and serializes them to bytes. Stereo payloads preserve the two channels; mono payloads are produced by averaging the channel samples. Each connected browser session has a queue and a channel preference. The capture thread schedules queue insertion onto the asyncio event loop using a thread-safe callback.

C. Network Layer

The aiohttp application serves the player files, exposes status and QR-code endpoints, and handles a WebSocket route. When a browser connects, the server reads a channel preference, registers a client session, waits for audio chunks, and sends each chunk as a binary WebSocket message. The server removes the session when the connection closes.

D. Client Playback Layer

The mobile page presents a large start-audio control, a volume slider, mono/stereo selection, jitter-buffer presets, and a keep-screen-awake option. The JavaScript player receives binary messages and schedules audio through the browser audio engine. The UI and code dossier describe continuous scheduled playback, visualizer support, and WakeLock/keep-alive behavior.

V. AUDIO FORMAT AND BANDWIDTH ANALYSIS

The configuration specifies a 44,100-Hz sample rate, 16-bit signed PCM, two capture channels by default, and a block size of 2,048 frames. The duration represented by one block is:

$$T_{block} = 2048 / 44100 = 0.04644\ s \approx 46.4\ ms.$$

For uncompressed PCM, payload bitrate is calculated as sample rate × bit depth × channel count. Therefore, stereo requires 44,100 × 16 × 2 = 1,411,200 bit/s (approximately 1.41 Mbit/s), while mono requires 44,100 × 16 × 1 = 705,600 bit/s (approximately 0.706 Mbit/s). These figures exclude WebSocket, TCP/IP, Bluetooth PAN, and link-layer overhead.

A stereo block contains 2,048 × 2 channels × 2 bytes = 8,192 bytes. A mono block contains 2,048 × 1 × 2 bytes = 4,096 bytes. At this block size, the nominal block production rate is 44,100 / 2,048 ≈ 21.53 blocks per second. Mono therefore halves the PCM payload relative to stereo and is the preferred bandwidth-saving mode for constrained links.

VI. IMPLEMENTATION DETAILS

A. Main Server (app.py)

The main application creates an aiohttp web application and registers routes for the mobile page, QR image, status response, and audio WebSocket. It binds to all host interfaces and attempts a sequence of ports beginning at the configured default port when a bind fails. A separate thread runs the server event loop, while the application starts the audio streamer and launches the Tkinter GUI unless command-line mode is selected.

B. Audio Engine (audio_streamer.py)

AudioStreamer stores the capture format, block size, pause state, connected-client set, lock, RMS value, worker thread, event loop, and selected speaker. It enumerates available speakers, starts a loopback capture worker, registers client queues, and unregisters sessions. If a client queue is full, the implementation removes the oldest queued item before inserting the new chunk. This policy favors freshness over preserving every sample, limiting stale backlog at the cost of possible audio discontinuity during congestion.

C. Network Configuration (config.py)

The configuration module defines the sample rate, channel count, block size, default port, host binding, and maximum queue size. Network discovery first attempts to inspect IPv4 adapter addresses through psutil and classifies Bluetooth interfaces separately from other adapters. A silent PowerShell query and hostname-based lookup are included as fallback approaches. The GUI prioritizes a discovered Bluetooth address when available.

D. Desktop GUI (gui.py)

The Tkinter dashboard displays the BlueBridge title, connection status, QR code, active IP selector, URL field, copy/open-browser controls, RMS meter, listener count, audio-source selector, pause/resume control, Bluetooth settings shortcut, and pairing guide. The GUI refreshes network information periodically and updates its meter on a short recurring UI callback.

E. Mobile Web Application

The static web application consists of HTML, CSS, JavaScript, a web manifest, and an icon. The mobile interface supports mono and stereo modes, a volume boost slider, three jitter-buffer presets labelled low-latency, smooth, and stable, and a screen-awake toggle. The browser must be on the same reachable local network as the PC and the user must initiate playback through the page control.

F. Optional Native Android Receiver

The Android Kotlin client searches the paired-device set for a device name containing PC or DESKTOP and otherwise selects the first paired device. It creates an RFCOMM socket using the standard SPP UUID 00001101-0000-1000-8000-00805F9B34FB, connects on a background thread, and writes received bytes to an AudioTrack configured for 44.1-kHz, stereo, 16-bit PCM. This implementation is an alternative receiver path that requires Android Bluetooth permissions and a compatible PC-side RFCOMM server.

VII. OPERATIONAL WORKFLOW

The browser-based Bluetooth PAN workflow is: (1) pair the phone and Windows PC in Bluetooth settings; (2) enable Bluetooth tethering or the platform's personal-hotspot capability on the phone; (3) connect the PC to the phone's Bluetooth access point through Windows PAN settings; (4) start BlueBridge using the supplied launcher or Python command; (5) select the Bluetooth IP address or scan the displayed QR code; (6) open the page on the phone and press START AUDIO; and (7) select mono or stereo and adjust the buffering and volume controls as needed. A shared Wi-Fi network can be used instead of Bluetooth PAN.

The native Android path is different: pair the devices, start the RFCOMM server, launch the Android receiver, establish the SPP socket, and play the incoming PCM stream through AudioTrack. The current Android code does not connect through the WebSocket endpoint.

VIII. PERFORMANCE ANALYSIS AND TEST STATUS

The supplied engineering dossier gives a latency budget of approximately 10–20 ms for capture, less than 1 ms for conversion/downmix, 3–8 ms for network transmission, 15–30 ms for browser buffering, and 5–10 ms for mobile output. Adding the minimum values gives approximately 34 ms and adding the maximum values gives approximately 69 ms. This is a component-level estimate in the source dossier, not a measured end-to-end result.

The source also includes a latency label in the mobile UI and marketing language referring to sub-30-ms performance. These statements are not supported by a supplied measurement procedure, synchronized timestamps, test logs, or device-specific results. Moreover, a single 2,048-frame capture block spans approximately 46.4 ms. Consequently, this report does not claim verified sub-30-ms end-to-end latency. A defensible performance claim requires measurement under controlled conditions.

A. Recommended Validation Procedure

- Record the PC output and phone output simultaneously using a common test signal, then estimate delay by waveform cross-correlation.
- Repeat at least 30 trials for each transport (Bluetooth PAN and Wi-Fi), channel mode (mono and stereo), and jitter-buffer preset.
- Record phone model, Android/iOS version, browser, Windows version, Bluetooth adapter, link type, RSSI where available, and ambient network load.
- Report median, 95th-percentile, minimum, and maximum latency, together with dropout counts and test duration.
- Measure CPU usage, memory use, queue depth, and audio underruns while increasing the number of connected clients.

Until those tests are performed, the implementation can be described as a low-latency-oriented design with configurable buffering, not as a system with a universally demonstrated latency value.

IX. MULTI-DEVICE CONNECTION: CURRENT SUPPORT AND FUTURE GOAL

The WebSocket server architecture already maintains a set of client sessions and creates a separate queue for each connected browser client. The desktop interface also displays a connected-phone count. These elements provide a foundation for more than one browser listener. However, the supplied material does not provide a measured multi-client capacity, synchronized playback guarantee, per-device administration interface, or stress-test results. The optional RFCOMM server shown in the dossier calls listen(1) and processes an accepted socket in a sequential loop; it should therefore not be represented as a completed multi-device RFCOMM service.

A future optional multi-device mode should allow a user to connect several phones simultaneously and select whether they receive the same stream or independent channel/buffer settings. Recommended development tasks are:

- Add an explicit client registry with stable session IDs, device labels, connection timestamps, transport type, channel mode, and playback state.
- Expose a listener-management panel in the desktop GUI, including connected/disconnected status and a per-client disconnect action.
- Retain per-client queues and add configurable queue limits, queue-depth telemetry, and a documented slow-client policy.
- Add an optional group mode that broadcasts the same source stream to all selected receivers.
- Introduce sequence numbers and capture timestamps in a small audio-frame header so receivers can estimate drift and align playback.
- Implement periodic clock-offset estimation and adaptive resynchronization for multi-room playback; avoid assuming that independent phone clocks are synchronized.
- Add configurable maximum-client limits and load tests to identify CPU, memory, network, and Bluetooth PAN bottlenecks.
- Keep RFCOMM multi-client support as a separate engineering task, potentially using one worker per client or an asynchronous Bluetooth framework where platform support permits.

The multi-device feature should be optional because each additional receiver increases network traffic and server work. For N receivers independently receiving the same PCM stream, the aggregate payload egress is approximately N times the single-client payload rate, before protocol overhead. For example, four stereo receivers would require approximately 5.64 Mbit/s of aggregate PCM payload. Actual sustainable capacity depends on the adapter, PAN implementation, radio conditions, and host networking stack.

X. LIMITATIONS AND SECURITY CONSIDERATIONS

- The primary implementation is Windows-oriented and depends on WASAPI loopback availability and compatible audio devices.
- Bluetooth PAN setup is platform- and driver-dependent; pairing alone does not necessarily establish a usable IP network.
- The server binds to 0.0.0.0, so it listens on all available IPv4 interfaces. The supplied design does not describe authentication, TLS, or access-control tokens.
- Uncompressed PCM consumes more bandwidth than a compressed codec such as Opus; codec support is listed as future work in the engineering dossier.
- Queue overflow handling drops the oldest frame, which can prevent growing delay but may produce audible gaps under sustained congestion.
- The optional Android code selects a paired PC heuristically and uses a simple byte-stream read/write loop; robust device selection, framing, reconnection, runtime permission handling, and lifecycle handling need further hardening.
- The supplied Launch.vbs example contains a machine-specific working directory and interpreter path and should be adapted before distribution.

For deployment beyond a trusted local network, BlueBridge should add a pairing token or one-time session code, restrict access to intended interfaces where possible, and consider HTTPS/WSS with certificate handling. The user should be informed that audio is transmitted on the local network and that any device

able to reach an unauthenticated server endpoint may attempt to connect.

XI. CONCLUSION

BlueBridge demonstrates a practical software architecture for turning a smartphone into a wireless audio endpoint for a Windows PC. Its principal browser path combines WASAPI loopback capture, NumPy PCM conversion, asynchronous WebSocket delivery, and Web Audio API playback. Bluetooth PAN removes the requirement for a shared Wi-Fi router while preserving an IP-based transport. The desktop dashboard simplifies connection setup through QR codes, IP discovery, source selection, and live audio-level monitoring. An optional Android SPP receiver provides a separate native Bluetooth route.

The supplied design specifies 44.1-kHz, 16-bit PCM with 2,048-frame blocks and offers mono mode to halve payload bandwidth. The source dossier's 34–69-ms latency budget is an estimate and should be validated experimentally before any strict latency claim is made. The WebSocket session model is a useful basis for optional multi-device support, while synchronized multi-room playback, explicit client management, capacity limits, and RFCOMM concurrency remain future engineering goals.

REFERENCES

[1] Microsoft, “WASAPI,” Microsoft Learn, Windows audio documentation. Available: https://learn.microsoft.com/windows/win32/coreaudio/wasapi. Accessed: Nov. 3, 2026.

[2] aiohttp contributors, “aiohttp: Asynchronous HTTP Client/Server for asyncio,” Project Documentation. Available: https://docs.aiohttp.org/. Accessed: Nov. 3, 2026.

[3] MDN Web Docs, “WebSocket API,” Mozilla. Available: https://developer.mozilla.org/docs/Web/API/WebSocket. Accessed: Nov. 3, 2026.

[4] MDN Web Docs, “Web Audio API,” Mozilla. Available: https://developer.mozilla.org/docs/Web/API/Web_Audio_API. Accessed: Nov. 3, 2026.

[5] Android Developers, “AudioTrack,” Android API Reference. Available: https://developer.android.com/reference/android/media/AudioTrack. Accessed: Nov. 3, 2026.

[6] Android Developers, “BluetoothSocket,” Android API Reference. Available: https://developer.android.com/reference/android/bluetooth/BluetoothSocket. Accessed: Nov. 3, 2026.

[7] BlueBridge Project, “High-Performance Audio Streamer: Turning Any Smartphone into a Real-Time PC Speaker,” Technical Dossier, System Design and Codebase Manual, version 2.0, 47 pp., supplied project documentation.

[8] BlueBridge Project source files: app.py, audio_streamer.py, config.py, gui.py, static/index.html, static/player.js, static/style.css, bluetooth_rfcomm.py, and android_client/MainActivity.kt, supplied code excerpts.

APPENDIX A. PROJECT SCREENSHOTS

The following figures are reproduced from the screenshots supplied with the project. They document the desktop control panel, Bluetooth setup guidance, mobile web player, Bluetooth PAN connection confirmation, and repository structure. They are implementation evidence, not substitutes for controlled performance measurements.



Fig. 2. Windows desktop control panel showing QR pairing, status, active IP, audio meter, and controls.

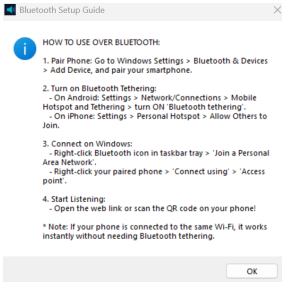


Fig. 3. Desktop Bluetooth pairing guide presented by the application.

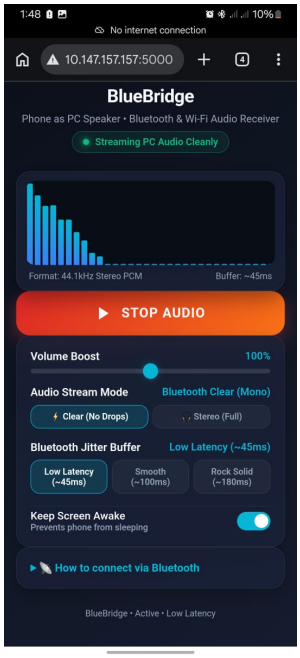


Fig. 4. Mobile web player displaying an active audio session and visualizer.

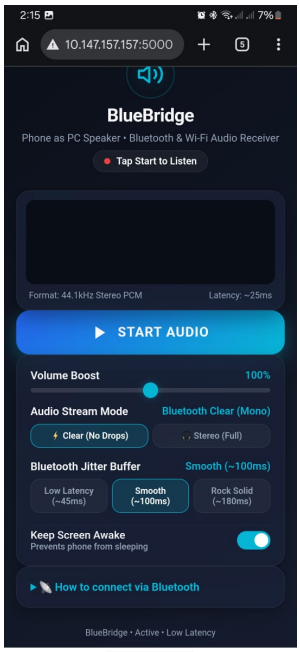


Fig. 5. Mobile web player before playback, showing stream and buffer controls.

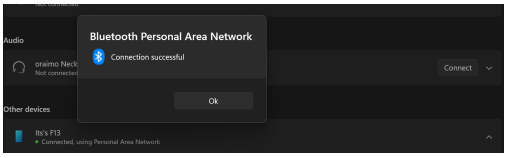


Fig. 6. Windows Bluetooth Personal Area Network connection confirmation.

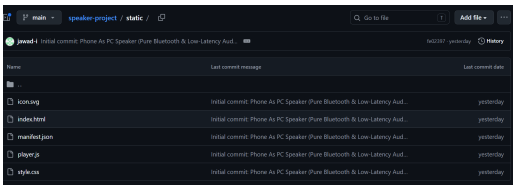


Fig. 7. Repository view showing the static web-player files.

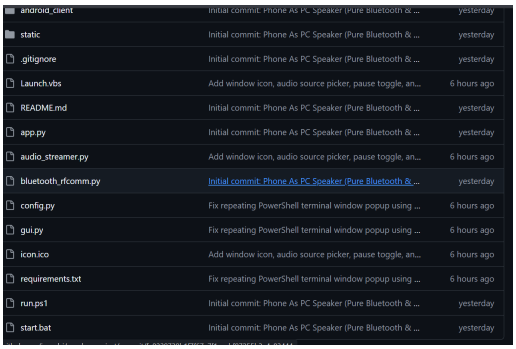


Fig. 8. Repository view showing the Python server, audio engine, configuration, GUI, and launch files.